SyncQuest

A Peer-to-Peer Collaborative Text Editor

Distributed Systems Project - Part 1

Real-time, server-free collaborative editing built on Yjs CRDTs and WebRTC data channels, wrapped in a gamified "realm and party" interface. This document explains the problem it solves, the distributed-systems concepts it demonstrates, its architecture, and how to run and use it.

Stack	React + Vite, Yjs, y-webrtc, y-quill, Quill.js, Node.js (ws)
Scope	Part 1 only -- no vector clocks, leader election, or audit log (Part 2)
Deployment	Signaling server runs standalone; frontend is a static Vite build

1. Problem Statement

Most collaborative editors (Google Docs, Notion, etc.) route every keystroke through a central application server that owns the document, arbitrates conflicts, and must stay available for the app to function at all. That central server is a single point of failure, a mandatory trust anchor, and -- in a classroom/offline/local-network setting -- often simply unnecessary infrastructure to stand up and pay for.

This project asks: **can two or more browsers edit one shared document in real time without any server ever holding, relaying, or arbitrating the document content itself?** The answer demonstrated here is yes -- using a Conflict-free Replicated Data Type (CRDT) for the document and a direct WebRTC connection between browsers for transport, with a server used only to help two browsers that don't know each other's network address find one another.

2. Use Cases

- **Classroom / lab demo of distributed systems concepts** -- CRDTs, eventual consistency, P2P networking, and NAT traversal, all observable in a live, editable app rather than only in theory.
- **Local-network collaborative notes** -- a team on the same Wi-Fi (e.g. a hackathon, a study group) can co-edit notes with zero backend to deploy or maintain beyond a tiny signaling relay.
- **Resilience demonstration** -- the signaling server can be killed mid-session and already-connected peers keep working, illustrating the difference between a coordination service and a data-plane dependency.
- **Foundation for Part 2** -- a working, tested P2P sync layer that vector clocks, leader election, and an audit log will be layered on top of, without needing to revisit the sync core.

3. Topics Covered

This project is deliberately built to exercise a specific set of distributed-systems concepts:

- **CRDTs (Conflict-free Replicated Data Types)** -- Yjs's `Y.Text`, a sequence CRDT built on a YATA-style algorithm, used for the document body.
- **Strong Eventual Consistency** -- every replica that has seen the same set of updates converges to an identical state, regardless of the order updates arrive in.
- **Peer-to-peer networking with WebRTC** -- direct browser-to-browser data channels, STUN-based NAT traversal, and the offer/answer/ICE handshake.
- **Signaling protocol design** -- a minimal pub/sub relay over WebSocket used purely for peer discovery, decoupled entirely from the application data plane.
- **Ephemeral distributed state (awareness)** -- presence and cursor position, which unlike the document itself does not need to be durable or CRDT-merged, only broadcast.
- **Local vs. distributed persistence** -- the deliberate distinction between `y-indexeddb` (per-browser durability across refresh) and a genuinely distributed/replicated store (out of scope for Part 1).
- **Client-server vs. peer-to-peer architecture trade-offs** -- demonstrated concretely by killing the signaling server mid-session and observing what does and doesn't break.

4. System Architecture

The system has exactly two kinds of process: a browser running the React/Yjs frontend, and a minimal Node.js signaling server. The diagram below distinguishes the handshake path (dashed, through the server) from the real data path (solid, direct between browsers):

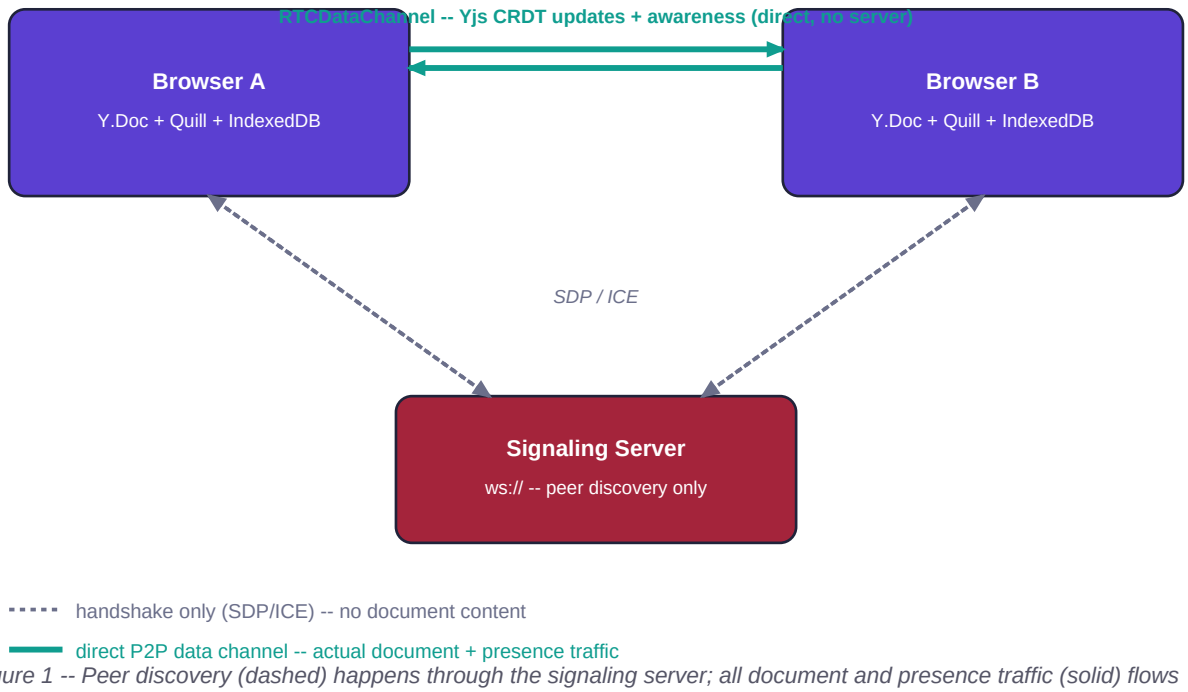

Figure 1 -- Peer discovery (dashed) happens through the signaling server; all document and presence traffic (solid) flows directly between browsers over an RTCDataChannel.

4.1 Components

- **Y.Doc** -- the CRDT document root; one shared `Y.Text` per room holds the rich-text content.
- **IndexeddbPersistence (y-indexeddb)** -- writes the document to the browser's local IndexedDB so a refresh doesn't lose unsynced edits. Local only -- not a distributed store.
- **WebrtcProvider (y-webrtc)** -- connects to the signaling server to discover peers, then opens a direct `RTCDataChannel` per peer using Google's public STUN server for NAT traversal.
- **QuillBinding (y-quill)** -- keeps the Quill rich-text editor and the shared `Y.Text` in sync in both directions, and renders remote peers' cursors/selections from awareness state.
- **Awareness (y-protocols/awareness)** -- ephemeral shared state (name, avatar, cursor position) broadcast over the same data channel, used for presence and live cursors.
- **Signaling server (Node.js + ws)** -- a small WebSocket relay. Its entire state is `Map<roomName, Set<socket>>`; it forwards opaque SDP/ICE envelopes between sockets subscribed to the same room and nothing else.

4.2 Why the signaling server does not break the "no central authority" claim

> **The boundary, explicitly:**
> "No central authority" here means no central authority over *document state or conflict resolution*. The signaling server never sees a single Yjs update -- its only job is analogous to a phone book, helping two parties find each other's network address. Once a data channel is open, document sync, CRDT merging, and presence all happen entirely peer-to-peer; killing the signaling server does not affect already-connected peers (verified by testing -- see Section 8).

4.3 How concurrent edits are resolved (no server arbitration)

Every character insertion into `Y.Text` is anchored to its immediate left/right neighbor *characters* (not numeric positions), and tagged with a `(clientID, clock)` pair. Because the anchor is a specific character object rather than an index, two peers can insert at "the same place" concurrently and, when each applies both operations in any order, they deterministically converge on the same final sequence -- this is Strong Eventual Consistency:

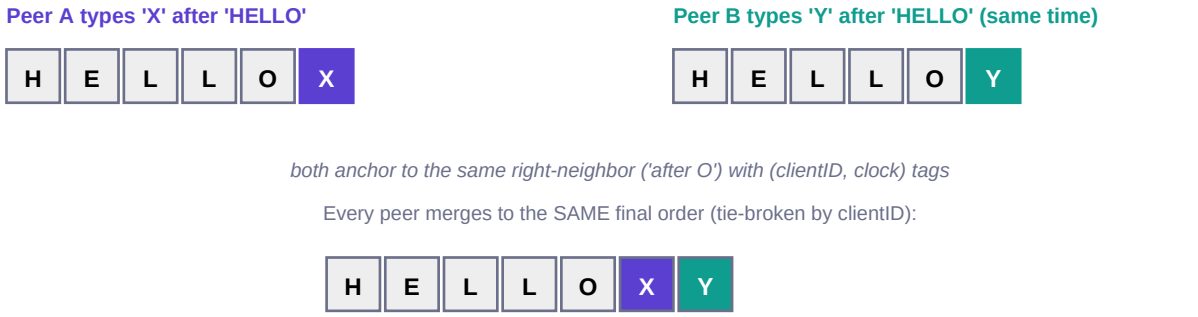

Figure 2 -- Two peers insert concurrently after the same character. Each operation carries a (clientID, clock) tag, so every replica applies a deterministic tie-break rule and converges on an identical final order -- with no negotiation and no server involved.

5. Gamified Interface - "SyncQuest"

The UI is themed as a shared "realm" the party is writing in together, purely as a cosmetic layer over the same functional requirements -- nothing here changes the underlying protocol, data model, or architecture described above.

- **Room to Realm**: the room ID becomes a "Realm code"; the shareable URL is a "Portal Link".
- **Connection status to Portal status**: "Opening the portal...", "Portal Open -- Synced!", "Portal Sealed" (error) -- same connecting/connected/error states as Part 1's requirements, reskinned.
- **Presence panel to Party panel**: each peer gets a random avatar emoji and fantasy "class" (Scribe, Sage, Ranger, ...), purely cosmetic fields alongside the real name/color used for cursor highlighting.
- **Quest Progress bar**: a shared "Realm Level" and XP bar computed live from the length of the actual shared `Y.Text` -- since every peer already has an identical view of that CRDT state, this number is automatically consistent across all peers with zero extra network traffic or protocol changes.
- **Toast notifications**: "Ally joined the realm!" / "Ally left the realm." on presence changes, plus a one-time "Achievement unlocked: Party Assembled!" toast on first successful peer connection.
- **Dark neon theme**: a pixel-display heading font (Press Start 2P) paired with a clean body font (Space Grotesk), glowing status badges, and a restyled Quill toolbar/editor to match.

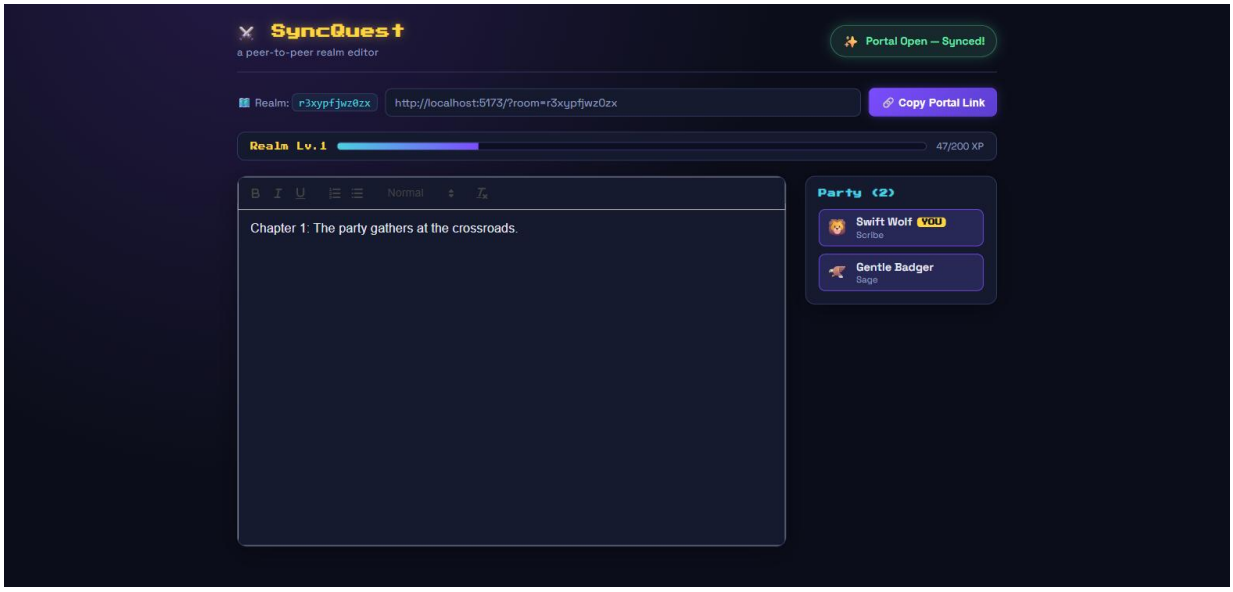

Figure 3 -- Two peers connected in the same realm; the XP bar (47/200) is identical on both screens because it's derived from the shared CRDT document length.

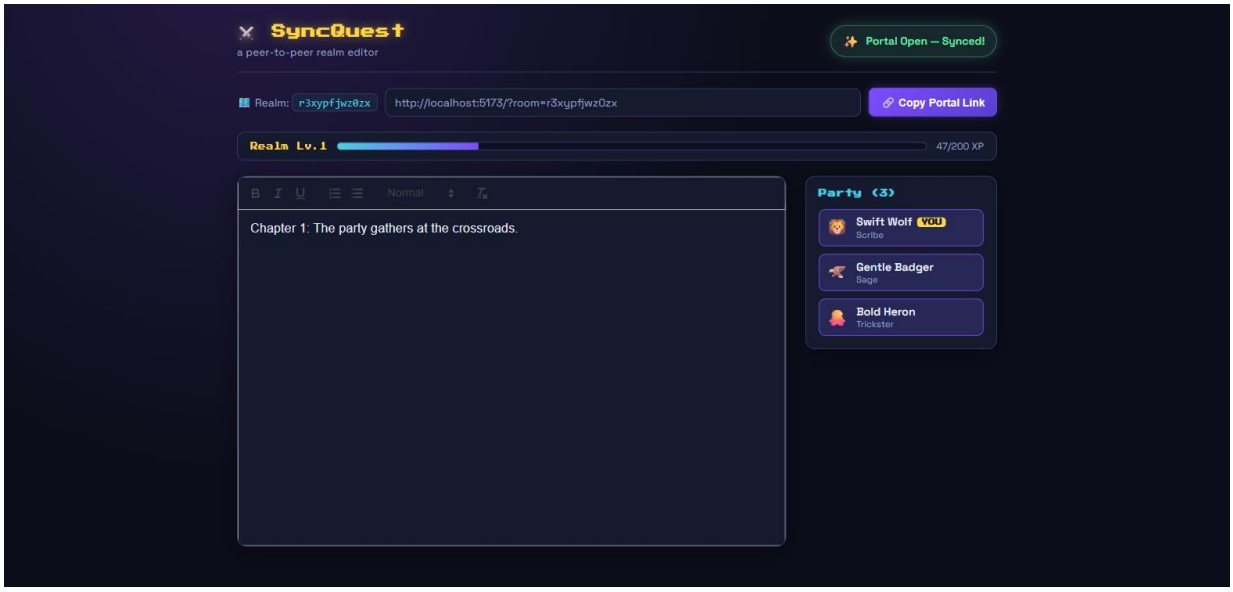

Figure 4 -- A third peer (Bold Heron, the Trickster) joins live; the Party panel updates to 3 members within seconds, with no crash or manual refresh.

6. How to Run the Project

Prerequisites: Node.js 18 or newer.

1. Install dependencies (from the `p2p-editor/` root):

```
cd p2p-editor
npm run install:all
```

2. Start the signaling server (Terminal 1):

```
npm run signaling
# listens on ws://localhost:4444
```

3. Start the frontend (Terminal 2):

```
npm run dev
# serves http://localhost:5173
```

Open `http://localhost:5173` -- a realm code is generated and appended to the URL automatically. Open the same URL in a second tab, an incognito window, or another device on the same Wi-Fi to test collaboration.

Testing across two real devices on the same Wi-Fi:

- Find the host machine's LAN IP (Windows: `ipconfig` then look for IPv4 Address).
- Set `VITE_SIGNALING_URL=ws://<that-ip>:4444` in `frontend/.env` and restart the dev server.
- On device 1, open `http://<that-ip>:5173` (not `localhost`) and copy the Portal Link.
- Open that link on device 2 -- both must be able to reach ports 5173 and 4444 on the host (allow the firewall prompt on first run).

Full details, deployment steps (Render/Docker), and the complete testing checklist are in `README.md` and `ARCHITECTURE.md` in the project folder.

7. How to Use the App

- Opening the app assigns you a random avatar, name, and "class" for this browser tab (persisted across refresh via `sessionStorage`, so refreshing doesn't rename you mid-session).
- The **Portal status badge** (top right) tells you at a glance whether you're still connecting, fully synced with peers, or unable to reach anyone.
- Click **Copy Portal Link** to grab a shareable URL for the current realm -- send it to anyone who should join.
- Type in the editor as you would in any rich-text app -- bold, italic, underline, and ordered/bullet lists are supported via the toolbar. Every keystroke syncs to all connected peers, typically within about a second on a local network.
- The **Party panel** (right side) lists everyone currently in the realm, live. Each remote peer's cursor and text selection are highlighted in their own color inside the editor.
- The **Quest Progress bar** fills as the shared document grows -- it's a fun read on collective progress, not a personal score.
- If you refresh the tab, your last-known document state reappears instantly (from local browser storage) while the app reconnects to peers in the background.

8. Testing and Verification Summary

The following were verified against the running app (not assumed) using automated browser control across multiple simultaneous tabs:

Check	Result
Two-tab real-time sync	Text appears cross-tab within ~1s
Three-tab sync (not just pairwise)	All three converge to identical text
Presence updates on tab close	Party count drops within seconds, no crash
Refresh recovery	Local content restored instantly, then re-syncs
Signaling server killed after connect	Already-connected peers keep syncing
New peer after signaling killed	Same-browser tab still finds peers via BroadcastChannel; a genuinely separate browse

Known limitations (intentionally out of scope for Part 1):

- No TURN server -- STUN-only NAT traversal, sufficient for a same-network demo but not guaranteed across restrictive/symmetric NATs.
- "Portal Sealed" (error) status is a timeout heuristic (about 8 seconds), since y-webrtc doesn't expose a single definitive "handshake failed" event.
- No vector clocks, leader election, or audit log -- reserved for Part 2, to be layered on top of this already-tested sync core.

9. Summary

SyncQuest is a fully working, tested demonstration that real-time collaborative document editing does not require a central server to own the document or resolve conflicts. Two or more browsers open a direct WebRTC connection -- after a lightweight signaling server helps them find each other -- and from that point on, all document updates and presence data flow peer-to-peer. Conflicts between concurrent edits are resolved automatically and deterministically by Yjs's CRDT algorithm, with no server-side arbitration step at all.

The gamified "realm and party" interface is a cosmetic skin over this same architecture: room becomes realm, presence becomes party, connection status becomes portal state, and a shared XP bar is derived -- read-only -- from the same CRDT state every peer already agrees on. None of this changes what data crosses the network or where it's stored.

Every functional requirement for Part 1 -- room-based sessions, real-time rich-text sync, live presence with remote cursors, a visible connection-status indicator, a signaling server that structurally cannot see document content, and local persistence across refresh -- was implemented and verified end-to-end, including the specific claim that already-connected peers keep working after the signaling server is killed. The system is ready to serve as the foundation Part 2 (vector clocks, leader election, audit log) will build on.

See `README.md` for full setup/testing instructions and `ARCHITECTURE.md` for the complete data-flow diagrams and CRDT explanation this report summarizes.